

AI Chat Assist - Technical Documentation

This document provides a technical overview of the AI Chat Assist application developed using Next.js, React, TypeScript, and AI integrations with Groq and Google Gemini.

1. Project Overview

AI Chat Assist is a modern AI-powered chat application designed to provide users with real-time conversational AI interactions using streaming responses similar to mainstream Generative AI products.

The application was designed with a focus on scalable architecture, reusable components, secure API communication, responsive design, and production-style frontend engineering practices.

2. Key Features

- Real-time AI chat with streaming responses
- Multi-conversation sidebar support
- Persistent conversation history using localStorage
- Dynamic message rendering without page refresh
- Loading states and user-friendly error handling
- Groq and Google Gemini AI integrations
- Automatic Groq fallback when Gemini rate limits occur
- Secure API communication through Next.js API routes
- Markdown rendering with sanitization support
- Copy message and copy code block functionality
- Edit and resend user prompts
- Delete and clear conversation actions (sidebar delete and header clear chat)
- Input validation for empty messages and character limits
- Responsive mobile and desktop layouts
- Unit testing with Vitest and React Testing Library

3. Technology Stack

Technology	Purpose
Next.js 16	Full-stack framework (UI + API routes)
React 19	UI component rendering
TypeScript	Static type safety
Tailwind CSS 4	Responsive application styling
Groq API	AI provider integration
Google Gemini API	AI provider integration
react-markdown	Markdown rendering
remark-gfm + rehype-sanitize	Markdown sanitization
localStorage	Conversation persistence

4. Application Architecture

The application follows a modular component-based architecture using the Next.js App Router. Frontend functionality is separated into reusable components, custom hooks, utility modules, and secure server-side API routes.

- Reusable UI component architecture
- Dedicated useChat hook for chat state management
- Server-side AI provider integration
- Streaming response handling using SSE
- JSON fallback support for reliability
- Persistent localStorage conversation management
- Type-safe application development using TypeScript
- Responsive mobile and desktop user interface

5. Project Structure

AI-Chat-Assist/

```
|
|
| └─ src/
|   |
|   | └─ app/
|   |   |
|   |   | └─ api/chat/route.ts
|   |   | └─ globals.css
|   |   | └─ layout.tsx
|   |   └─ page.tsx
|   |
|   | └─ components/
|   | └─ hooks/
|   | └─ lib/
|   |   |
|   |   | └─ ai/      (groq, gemini, provider, stream-events)
|   |   | └─ stream/  (parse-sse)
|   |   | └─ types.ts
|   |   | └─ conversations.ts
|   |   | └─ storage.ts
|   |   └─ ...
|   |
|   └─ test/setup.ts
|
| └─ public/
|
| └─ vitest.config.ts
|
| └─ next.config.ts
```

```
├── .env.example
├── package.json
└── README.md
```

The project structure is organized to separate frontend UI logic, AI provider integrations, stream handling, reusable utilities, and testing configuration for better scalability and maintainability.

6. Environment Configuration

AI_PROVIDER=groq

GROQ_API_KEY=your_groq_api_key
GROQ_MODEL=llama-3.3-70b-versatile

GEMINI_API_KEY=your_gemini_api_key
GEMINI_MODEL=gemini-2.5-flash

The application supports both Groq and Google Gemini AI providers. The active provider is controlled using the AI_PROVIDER environment variable.

Configuration	Primary Provider
AI_PROVIDER=groq	Groq
AI_PROVIDER=gemini	Gemini
Both API keys configured without AI_PROVIDER	Gemini
Only GROQ_API_KEY configured	Groq

When Gemini is configured as the primary provider and rate-limit or quota-limit errors occur, the application automatically retries requests using Groq if a Groq API key is available.

All API credentials are securely managed using server-side environment variables and are never exposed in browser-side code.

7. Installation and Running the Application

Repository URL: <https://github.com/nikshithareddyaela/AI-Chat-Assist>

Prerequisites:

- Node.js 18+
- npm 9+

1. Clone the repository from GitHub
2. Navigate to the project directory
3. Install dependencies using npm install

4. Copy `.env.example` to `.env.local`
5. Configure environment variables in `.env.local`
6. Start the development server using `npm run dev`

Command	Description
<code>npm run dev</code>	Start development server
<code>npm run build</code>	Create production build
<code>npm run start</code>	Run production build
<code>npm run lint</code>	Run ESLint
<code>npm run test</code>	Run unit tests
<code>npm run test:watch</code>	Run tests in watch mode

8. Deployment

The application is configured for deployment on Vercel. Environment variables configured locally should also be configured within the Vercel deployment environment.

Live Demo: <https://ai-chat-assist-mu.vercel.app>

9. Coding Standards and Best Practices

- Component-based architecture
- TypeScript-based type safety
- Reusable and maintainable code structure
- Semantic HTML implementation
- Responsive Tailwind CSS layout
- Secure server-side API handling
- Environment variable protection
- Input validation and user-facing error handling
- Streaming response implementation with fallback support
- Sanitized markdown rendering
- Unit testing implementation

10. Future Enhancements

- User authentication support
- Conversation export functionality
- Request cancellation while streaming
- Enhanced accessibility support
- Cloud database synchronization
- Dark/light theme toggle

11. Conclusion

AI Chat Assist demonstrates modern frontend engineering practices through scalable architecture, streaming AI integrations, reusable components, secure API handling, responsive

UI design, and robust state management. The project emphasizes maintainability, reliability, and production-oriented development standards.